

Podstawowe funkcje biblioteki narzędziowej „Neural Network Toolbox. Version 5” pakietu MATLAB v. 6

I. Funkcje przeznaczone do tworzenia jednokierunkowej sieci neuronowej

- `newff` - tworzenie wielowarstwowej jednokierunkowej sieci neuronowej,
- `newp` - tworzenie sieci jednowarstwowej złożonej z perceptronów.
- `newlin` - tworzenie sieci jednowarstwowej złożonej z neuronów liniowych.
- `newlind` - projektowanie sieci jednowarstwowej złożonej z neuronów liniowych.

II. Funkcje przeznaczone do uczenia sieci neuronowych (uniwersalne)

- `train` - funkcja realizuje trening sieci neuronowej (dowolnej),
- `adapt` - funkcja realizuje jeden krok (epokę) treningu sieci neuronowej.

III. Funkcje przeznaczone do symulacji sieci neuronowych

- `sim` - funkcja wyznacza wartości wyjść sieci neuronowej.

IV. Inne funkcje dedykowane do prac z sztucznymi sieciami neuronowymi

- `plotpv` - funkcja wykreśla położenie dla perceptronu punktów oczekiwanych odpowiedzi
- `plotpc` - funkcja wykreśla granicę decyzyjną w przestrzeni wag sieci neuronowej

Część I. Funkcje przeznaczone do tworzenia jednokierunkowej sieci neuronowej

Newff

Funkcja tworzy wielowarstwową sieć neuronową - jej każda warstwa składa się z zadanej liczby neuronów o zadanych funkcjach aktywacji (liniowych bądź nieliniowych).

`net = newff(pr, [s1 s2...sn], {tf1 tf2...tfn}, btf, blf, pf)`

wejscia

- `pr` - macierz (o rozmiarze $R \times 2$) zawierająca min i max dla R wejść sieci
- `si` - liczba neuronów w i -tej warstwie sieci; liczba warstw wynosi n ;
- `tfi` - funkcja aktywacji neuronów i -tej warstwy sieci, domyślnie = 'tansig';
- `btf` - nazwa funkcji wykorzystywanej do treningu sieci; domyślnie = 'trainlm' ;
- `blf` - nazwa funkcji używanej do modyfikacji wag; domyślnie = 'learnp';
- `pf` - określenie funkcji celu; domyślnie = 'mse'.

wyjście

- `net` - obiekt opisujący architekturę n -warstwowej sieci neuronowej .

Opis szczegółowy parametrów wejściowych

Parametr `pr` określa z jakiego przedziału wartości mogą pojawić się na poszczególnych wejściach sieci, innymi słowy informuje jaką najmniejszą i największą wartość mogą mieć dane podawane na poszczególne wejścia sieci. Kolejne wektory wejściowe powinny tworzyć kolumny macierzy wejściowej – wówczas we wierszach będą znajdować się kolejne wartości jakie mają być podane na sieć. Parametr `pr` najwygodniej jest wyznaczyć używając funkcji `minmax`* w której jako argument podać należy macierz wektorów wejściowych sieci.

Sieć może być zbudowana z dowolnej liczby warstw. Każda warstwa może mieć inną ilość neuronów, którą określamy w wektorze `[s1 s2...sn]`. Pamiętać należy, iż ostatni element wektora (parametr `sn`) określa liczbę wyjść całej sieci - musi być ona równa długości pojedynczego wektora oczekiwanych odpowiedzi sieci.

Funkcje przejścia (aktywacji) dla neuronów z poszczególnych warstw sieci (parametry `{tf1 tf2...tfn}`) mogą być różne - dla każdej warstwy sieci funkcję przejścia określa odrębny parametr. W ramach jednej warstwy wszystkie neurony mają taką samą funkcję przejścia. Dopuszczalne są funkcje:

- `tansig` - funkcja tangensoidalna,
- `logsig` - funkcja sigmoidalna,
- `purelin` - funkcja liniowa,
- `hardlim` - funkcja unipolarna,
- `hardlims` - funkcja bipolarna,
- `satlin` - funkcja liniowa z nasyceniem ,

* opis funkcji `minmax` znajduje się na stronie

satlins – funkcja liniowa symetryczna z nasyceniem.

Pamiętać należy, iż metody uczenia bazujące na wstecznej propagacji błędu wymagają aby funkcja przejścia była funkcją różniczkowalną.

Dostępnych jest wiele funkcji trenujących sieci jednokierunkowe (parametr **bt f**):

- traingd** - metoda gradientu prostego (wstecznej propagacji błędu),
- traingdm** - metoda propagacji wstecznej błędu z momentum,
- traingda** - metoda propagacji wstecznej błędu z adaptacyjną zmianą współczynnika uczenia,
- traingdx** - metoda propagacji wstecznej błędu z momentum i adaptacją współczynnika,
- trainlm** - metoda Levenberga-Marquardta,
- trainbr** - odmiana metody Levenberga-Marquardta z regularyzacją Bayes’owską,
- trainbfg** - metoda gradientowa Broydena-Fletcher-Goldfarba-Shano (quasi-Newtonowska),
- trainscg** - metoda skalowanego gradientu sprzężonego,
- traincgb** - metoda gradientu sprzężonego Powella-Beale’go,
- traincgf** - metoda gradientu sprzężonego Fletchera-Powella,
- traincgp** - metoda gradientu sprzężonego Polaka-Ribiere,
- trainrpf** - metoda propagacji wstecznej błędu RPROP (metoda Riedmiller, Brauna)

Funkcją uczącą wykorzystywaną do korekty wag podczas uczenia sieci (parametr **b l f**) może być jakakolwiek gradientowa funkcja ucząca np.:

- learnngd** - gradient prosty,
- learnngdm** - gradient prosty z momentum.

Parametr **pf** służy do określenia funkcji (tzw. funkcji celu) wykorzystywanej do wyznaczenia jakości wytrenowania jednokierunkowej sieci neuronowej. W zależności od rodzaju zastosowanej funkcji poszczególne składniki błędu mają różny wpływ na jej wartość, a co za tym idzie na ocenę jakości wyszkolenia sieci. Predefiniowane są następujące funkcje celu:

- mse** - błąd średniokwadratowy,
- msereg** - suma błędu średniokwadratowego i kwadratów wag sieci,
- sse** - suma kwadratów błędów.

Parametrem może być dowolna różniczkowalna funkcja błędu.

Opis szczegółowy generowanego obiektu **net**

architektura:

- numInputs**: 1 – liczba wejść sieci
- numLayers**: 2 – liczba warstw sieci
- biasConnect**: [1; 1] – określa czy warstwa ma bias (1-tak, 0-nie)
- inputConnect**: [1; 0]
- layerConnect**: [0 0; 1 0]
- outputConnect**: [0 1]
- targetConnect**: [0 1]

numOutputs: 1 (read-only)
numTargets: 1 (read-only)
numInputDelays: 0 (read-only)
numLayerDelays: 0 (read-only)

subobject structures:

inputs: {1x1 cell} of inputs
layers: {2x1 cell} of layers
outputs: {1x2 cell} containing 1 output
targets: {1x2 cell} containing 1 target
biases: {2x1 cell} containing 2 biases
inputWeights: {2x1 cell} containing 1 input weight
layerWeights: {2x2 cell} containing 1 layer weight

funkcje:

adaptFcn: 'trains'
initFcn: 'initlay'
performFcn: 'mse'
trainFcn: 'trainlm'

parameters:

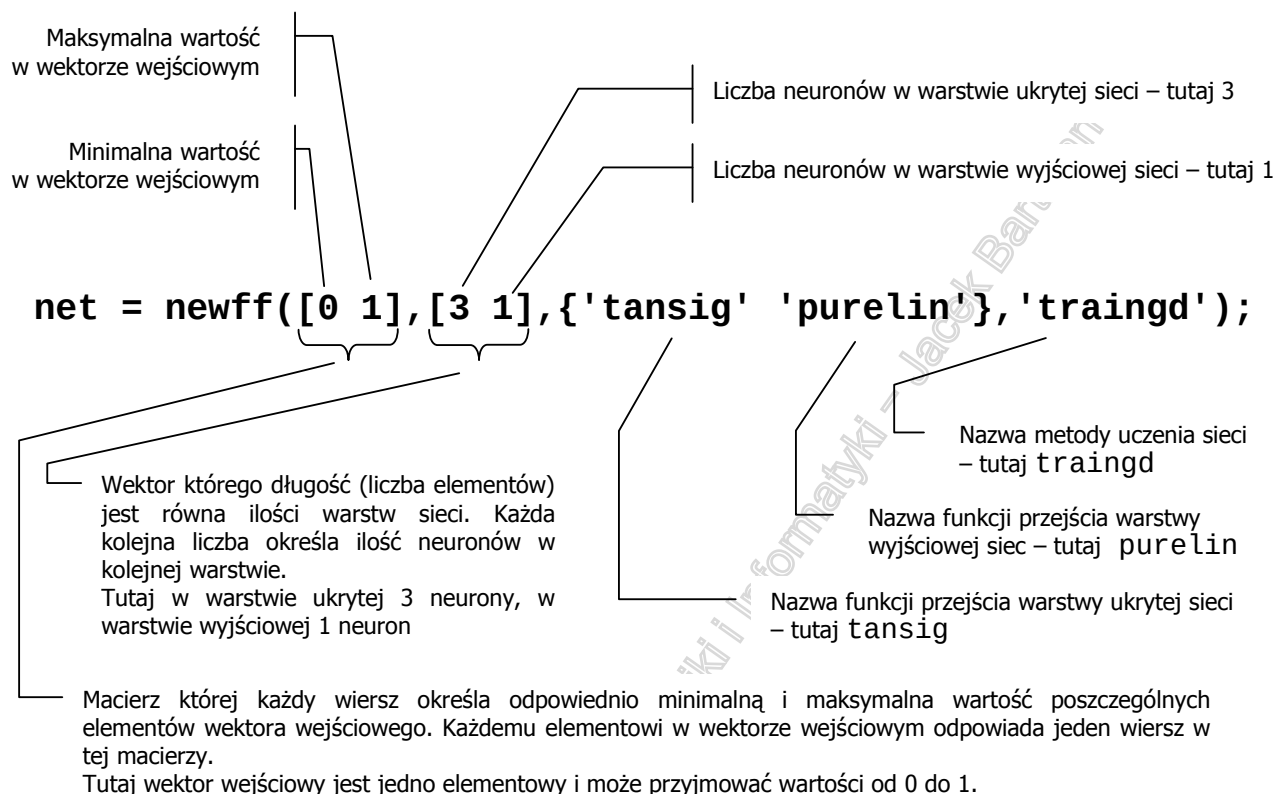
adaptParam: .passes
initParam: (none)
performParam: (none)
trainParam: .epochs, .goal, .max_fail, .mem_reduc,
.min_grad, .mu, .mu_dec, .mu_inc,
.mu_max, .show, .time

weight and bias values:

IW: {2x1 cell} containing 1 input weight matrix
LW: {2x2 cell} containing 1 layer weight matrix
b: {2x1 cell} containing 2 bias vectors

other:

userdata: (user stuff)

Przykład:

Wydanie powyższego polecenia spowoduje utworzenie sieci jednokierunkowej dwuwarstwowej z jednym wejściem (podano tylko jedną parę określającą zakres wektora wejściowego – parametr `pr`). W warstwie ukrytej będzie 3 neurony z funkcjami aktywacji `tansig` a w warstwie wyjściowej 1 neuron z funkcją aktywacji `purelin`. Sieć będzie uczona klasyczną metodą wstecznej propagacji błędów `traingd`. Pozostałe parametry przyjmą wartości domyślne – do treningu sieci zostanie wykorzystana funkcja `'trainlm'`, do modyfikacji wagi funkcja `'learnp'`, a jakość wytrenowania sieci będzie oceniana w oparciu o funkcję `'mse'`. Tak utworzona sieć jest obiektem o nazwie `net`, którego poszczególne atrybuty możemy zmieniać używając wprost operacji podstawienia np.:

`net.trainParam.epochs = 5000` – ustali maksymalną liczbę epok uczenia na 5000.

newp

Funkcja tworzy jednowarstwową sieć neuronową, złożoną z zadanej liczby perceptronów (sztucznych neuronów z bipolarną lub unipolarną funkcją przejścia).

net = newp(pr, s, tf, lf)

wejścia

- `pr` - macierz (o rozmiarze $R \times 2$) zawierająca min i max dla R wejść sieci
- `s` - liczba neuronów w sieci;
- `tf` - funkcja aktywacji neuronów, domyślnie = `'hardlim'`,

lf - nazwa funkcji używanej do modyfikacji wag; domyślnie = 'learnp';
 wyjście
net - obiekt opisujący architekturę utworzonej sieci neuronowej

Funkcją aktywacji perceptronów (parametr **tf**) oprócz funkcji unipolarnej (**hardlim**) może być funkcja unipolarna (**hardlims**). Jako parametru **lf** możemy użyć nazw następujących funkcji uczących:

learnp - modyfikacja wag w oparciu o regułę perceptronowi,
learnpn - modyfikacja wag w oparciu o regułę perceptronową (szybsza gdy wektor wejściowy zawiera elementy o bardzo różnych wartościach).

Przykład:

```
net = newp([-1 1; 2.5 4], 1)
```

Podane powyżej wywołanie funkcji tworzy sieć zawierającą pojedynczy neuron o dwóch wejściach. Zakres wartości pierwszego wejścia to [-1, 1] drugiego [2.5, 4]. Wywołanie funkcji zawiera tylko dwa argumenty – pozostałe dwa przyjmują wartości domyślne: **hardlim** dla funkcji aktywacji neuronu oraz **learnp** dla funkcji korekty wag sieci:

newlin Funkcja tworzy jednowarstwową sieć neuronową, złożoną z zadanej liczby neuronów o liniowych funkcjach aktywacji. Tego typu sieć jest zwykle wykorzystywana jako filtr adaptacyjny do przetwarzania sygnałów lub predykcji szeregów czasowych.

```
net = newlin(pr, s, id, lr)
```

wejścia

pr - macierz (o rozmiarze Rx2) zawierająca min i max dla R wejść sieci;
s - liczba neuronów w sieci (równa ilości wyjść z sieci);
id - wejściowy wektor opóźnień, domyślnie = [0] ;
lr - współczynnik uczenia sieci; domyślnie = 0.001.

wyjście

net - obiekt opisujący architekturę utworzonej sieci neuronowej.

```
net = newlin(pr, s, 0, P)
```

wejścia

pr - macierz (Rx2) zawierająca min i max wartości dla R wektorów wejściowych;
s - liczba neuronów w sieci (równa ilości wyjść z sieci);
0 - wartość zero;

P - macierz wektorów wejściowych (każda kolumna macierzy to jeden wektor wejściowy).

wyjście

net - obiekt opisujący architekturę utworzonej sieci neuronowej

Przykład:

```
net = newlin([-100 100], 2, [0 0], 0.01);
```

Przedstawione powyżej wywołanie funkcji tworzy sieć liniową zawierającą dwa neurony o jednym wejściu. Zakres wartości wejściowej to [-100, 100], opóźnienie sygnału dla pierwszego wejścia wynosi 0, dla drugiego 0 (elementy wektora **P1** są w tym przypadku traktowane jako kolejne próbki sygnału). Założona wartość stałej szybkości uczenia wynosi 0.01.

newlind Funkcja tworzy jednowarstwową sieć neuronową, złożoną z zadanej liczby neuronów o liniowych funkcjach aktywacji. Tego typu sieć jest zwykle wykorzystywana jako filtr adaptacyjny do przetwarzania sygnałów lub predykcji szeregów czasowych.

Ponadto funkcja oblicza wagi **w** oraz wartości bias (progi) **b** neuronów sieci w wyniku rozwiązania metodą najmniejszych kwadratów równania liniowego:

$$[\mathbf{w} \ \mathbf{b}] * [\mathbf{P}; \text{ones}] = \mathbf{T}$$

```
net = newlind(P, T, pi)
```

wejścia

P - macierz wektorów wejściowych (każda kolumna macierzy to jeden wektor wejściowy)*,

T - macierz pożądanych odpowiedzi sieci (każda kolumna macierzy to jeden wektor oczekiwanych odpowiedzi)*,

Pi - macierz o wymiarach $I \times ID$, zawierająca początkowe wartości opóźnionych wejść sieci (ID jest liczbą opóźnień); jest to parametr opcjonalny, domyślnie **Pi** = []

wyjście

net - obiekt opisujący architekturę utworzonej sieci neuronowej

* Szczegółowy opis parametrów **P** oraz **T** umieszczony w opisie funkcji **train** na str. 9

- Ai** - macierz utworzona z wektorów początkowych opóźnień próbek sygnału w kolejnych warstwach sieci; domyślnie = 0,
- VV** - dane służące do przeprowadzenia procesu walidacji (oceny) treningu; macierze struktury walidacyjnej: VV.P, VV.T, VV.Pi, VV.Ai odpowiadają argumentom w podstawowej postaci wywołania funkcji treningu, tj.: P, T, Pi, Ai. Argument ten może przyjmować wartość pustą, [],
- TV** - dane służące do przeprowadzenia procesu testowania sieci po zakończeniu treningu; macierze struktury testowej: TV.P, TV.T, TV.Pi, TV.Ai odpowiadają argumentom w podstawowej postaci wywołania funkcji treningu, tj.: P, T, Pi, Ai. Argument ten może przyjmować wartość pustą, [].

wyjścia

- net** - obiekt opisujący architekturę nauczonej sieci neuronowej,
- tr** - informacja o przebiegu treningu (liczba epok uczenia, przebieg funkcji błędu itp.),
- y** - wartości wyjść wytrenowanej sieci,
- e** - wektory błędów popełnianych przez sieć podczas procesu uczenia.
- Pf** - wyjściowa macierz utworzona z wektorów opóźnień próbek sygnału wejściowego sieci (również uzyskana w procedurze adaptacji),
- Af** - wyjściowa macierz utworzona z wektorów opóźnień próbek sygnału w poszczególnych warstwach sieci (również uzyskana w procedurze adaptacji)

Opis szczegółowy parametrów wejściowych

Macierz **P** jest zbiorem **Q** wektorów kolumnowych o długości **R** – posiada więc wymiar $R \times Q$, np.:

$$P = \begin{bmatrix} 3 & 5 & 8 & 6 & 4 \\ -4 & -5 & -3 & -5 & -2 \\ -1 & 2 & 0 & 1 & 0 \end{bmatrix}$$

Pojedynczy wektor wejściowy

Podana powyżej macierz **P** składa się z 5 wektorów wejściowych, każdy złożony z 3 elementów – oznacza to, iż projektowana sieć będzie miała 3 wejścia.

Macierz oczekiwanych odpowiedzi **T** składa się z **Q** wektorów o długości **S** każdy – posiada więc wymiar $S \times Q$. Długość pojedynczego wektora wyjściowego (**S**) odpowiada liczbie neuronów w warstwie wyjściowej sieci. Natomiast ilość wektorów wyjściowych (**Q**) musi być równa ilości wektorów wejściowych, gdyż dla każdego wektora wejściowego - podczas uczenia nadzorowanego - musi być podana oczekiwana odpowiedź (wektor oczekiwanych odpowiedzi). Macierz odpowiedzi **T** dla podanej powyżej macierzy wejściowej **P** może mieć postać:

$$T = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & -1 & 0 & -1 & -1 \end{bmatrix}$$

Pojedynczy wektor wyjściowy

Zaznaczony wektor odpowiedzi (w obwódce) stanowi oczekiwaną odpowiedź sieci po podaniu na wejście 4 wektora wejściowego z macierzy **P** (w obwódce).

Przykład I

```
net=train(net,P,T)
```

Najprostsze i najczęściej stosowane wywołanie funkcji uczącej sieć neuronową. Wcześniej konieczne jest zadeklarowanie macierzy wektorów wejściowych – **P**, macierzy oczekiwanych odpowiedzi **T** oraz utworzenie sieci neuronowej – **net** (np.: przy pomocy funkcji **newff**). Sieć będzie uczona metodą zapisana w polu **net.trainFcn** (domyślnie będzie to metoda Levenberga-Marquardta), pozostałe parametry uczenia będą zgodne z zapisanymi w **net.trainParam** – będzie tam określony współczynnik uczenia, końcowa pożądana wartość błędu, maksymalna liczba epok uczenia itp.

Przykład II

Postawmy zadanie polegające na aproksymacji przez sieć neuronową funkcji **SINUS**. przedziale $[-\pi, \pi]$. Sieć będzie uczona w punktach równomiernie rozłożonych w całym przedziale, oddalonych od siebie o $\pi/30$. Efekty uczenia sprawdzimy zagęszczając przedział i rysując wykres funkcji **SINUS** oraz jej aproksymacji siecią neuronową.

```
P=[-pi:pi/30:pi];           % macierz wejściowa
T=sin(P);                   % macierz oczekiwanych odpowiedzi
% utworzenie sieci jednokierunkowej złożonej z dwóch warstw – w pierwszej
% 5 neuronów z funkcją przejścia 'tansig', w wyjściowej 1 neuron z
% liniową funkcją przejścia 'purelin'. Sieć uczona będzie metoda LM
net = newff([-1 1], [5 1], {'tansig' 'purelin'}, 'trainlm');
net.trainParam.epochs = 500; % uczenie będzie trwać max 500 epok
net.trainParam.goal = 0.001; % sieć jest nauczona gdy SSE ≤ 0.0001
net = train(net, P, T);      % uczenie sieci
P2=[-pi*2*pi/100:pi];       % zdefiniowanie zbioru testowego
y2 = sim(net, P2)            % symulacja pracy sieci – patrz str. 13
figure
% wykreślenie przebiegu funkcji sinus (na niebiesko) z naniesionymi
% punktami uczenia oraz uzyskanych odpowiedzi sieci (na czerwono)
plot(P, T, 'bo', P2, y2, 'r')
```

adapt Funkcja realizuje jeden krok (tzw. epokę) treningu sieci neuronowej, dokonując adaptacji wag synaptycznych sieci. Do adaptacji jest wykorzystywana funkcja, której nazwa została podana w polu **net.adaptFcn**, zaś wartości niezbędnych parametrów tej funkcji – w polu **net.adaptParam**. Funkcja ma taką samą postać dla wszystkich sieci.

wejścia

- wviście

-

wejścia

- net** - struktura (obiekt) zawierająca opis wielowarstwowej sieci jednokierunkowej
- P** - macierz wektorów wejściowych (każda kolumna macierzy to jeden wektor wejściowy),
- P_i** - macierz utworzona z wektorów początkowych opóźnień próbek sygnału wejściowego sieci; domyślnie = 0
- A_i** - macierz utworzona z wektorów początkowych opóźnień próbek sygnału w kolejnych warstwach sieci; domyślnie = 0
- T** - macierz pożądanych odpowiedzi sieci (każda kolumna macierzy to jeden wektor oczekiwanych odpowiedzi)

- y - wartości wyjść wytrenowanej sieci dla wektorów zapisanych w P,
- Pf - wyjściowa macierz utworzona z wektorów opóźnień próbek sygnału wejściowego sieci,
- Af - wyjściowa macierz utworzona z wektorów opóźnień próbek sygnału w poszczególnych warstwach sieci,
- e - wektory błędów popełnianych przez sieć podczas procesu uczenia,
- Perf - wartość funkcji oceny błędu odwzorowania sieci, której nazwę zawiera pole `'net.performFcn'` (domyślnie `'mse'` – błąd średniokwadratowy)

```
net = newp[0 1; 0 1],1); %utworzenie perceptronu o 2 wejściach i 1 wyjściu
disp(net)
% Symulacja pracy perceptronu dla trzech różnych wektorów uczących:
% (pokazano różne sposoby podwania danych)
p1 = [.2; .9]; a1 = sim(net, p1)
p2 = [.2 .5 .1; .9 .3 .7]; a2 = sim(net, p2)
p3 = {[.2; .9] [.5; .3] [.1; .7]}; a3 = sim(net, p3)
```

```
net = newlin([0 2; 0 2; 0 2], 2, [0 1]); % utworzenie sieci liniowej  
p1 = {[2; 0.5; 1] [1; 1.2; 0.1]};  
[y1, pf] = sim(net, p1) % obliczenie wyjść sieci dla dwóch  
                           %zadanych wektorów wejściowych (przy założeniu  
                           %zerowych opóźnień na wejściach):  
  
p2 = {[0.5; 0.6; 1.8] [1.3; 1.6; 1.1] [0.2; 0.1; 0]};  
[y2, pf] = sim(net, p2, pf) % Wywołanie wykorzystuje wartości opóźnień,  
                             %wyznaczone w poprzednim wywołaniu funkcji:
```

[illegible]

plotpv(P, T)

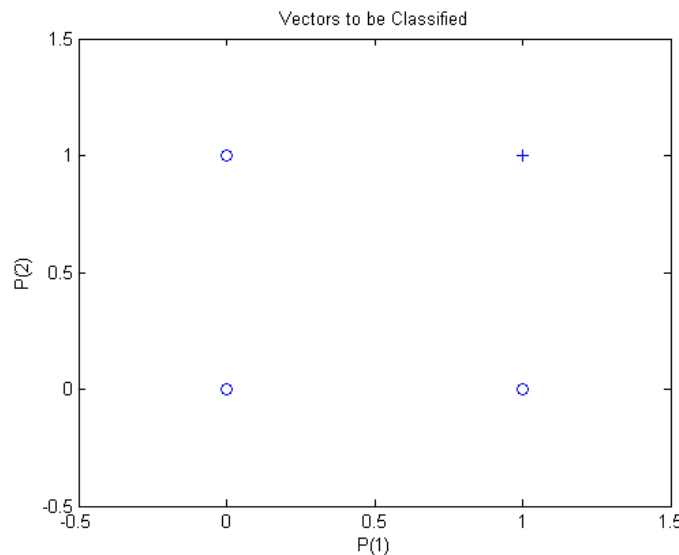
P - macierz wektorów wejściowych (każda kolumna macierzy to jeden wektor wejściowy),

- T** - macierz pożądaných odpowiedzi sieci (każda kolumna macierzy to jeden wektor oczekiwanych odpowiedzi)

Przykład:

Przykład pokazuje zdefiniowanie reguł bramki AND i wykreślenie uzyskanych danych na płaszczyźnie

```
p=[0 0 1 1
    0 1 0 1]; % macierz wektorów wejściowych
t=[0 0 0 1]; % macierz oczekiwanych odpowiedzi
plotpv(p,t);
```



plotpc Funkcja wykreśla granicę decyzyjną w przestrzeni wag sieci neuronowej, złożonej z perceptronów. Wywołanie funkcji ma sens, gdy wymiarowość wektora wejściowego sieci jest nie większa niż trzy.

line = plotpc (w, b, line)

wejścia

- w** - macierz wag synaptycznych perceptronu (o wymiarach $S \times R$, gdzie S jest liczbą neuronów w warstwie, zaś R – wymiarowością wektora wejść sieci, $R > 3$)
- b** - wektor wartości progów perceptronu (o wymiarach $S \times 1$)
- line** - nazwa ostatnio narysowanej linii (opcjonalne)

wejścia

- line** - nazwa rysowanej linii (opcjonalne). W przypadku wykorzystania funkcji w „pętli”, dzięki użyciu tego parametru kolejne wywołania funkcji usuwają poprzednio wyrysowaną linię.

Przykład:

Instrukcje poniżej definiują wejścia i wyjścia perceptronu i ukazują je na płaszczyźnie a następnie po zainicjowaniu sieci wykreślają granice decyzyjną

```
p = [0 0 1 1; 0 1 0 1];           % macierz wektorów wejściowych
t = [0 1 1 1];                   % macierz oczekiwanych odpowiedzi
plotpv(p, t)                     % wykreslenie punktów na płaszczyźnie
net = newp(minmax(p), 1);        % inicjacja sieci
net.iw{1,1} = [-1.2 -0.5];       % określenie wag
net.b{1} = 1;                    % określenie biasu
plotpc(net.iw{1,1}, net.b{1})    % wykreslenie granicy decyzyjnej
```

